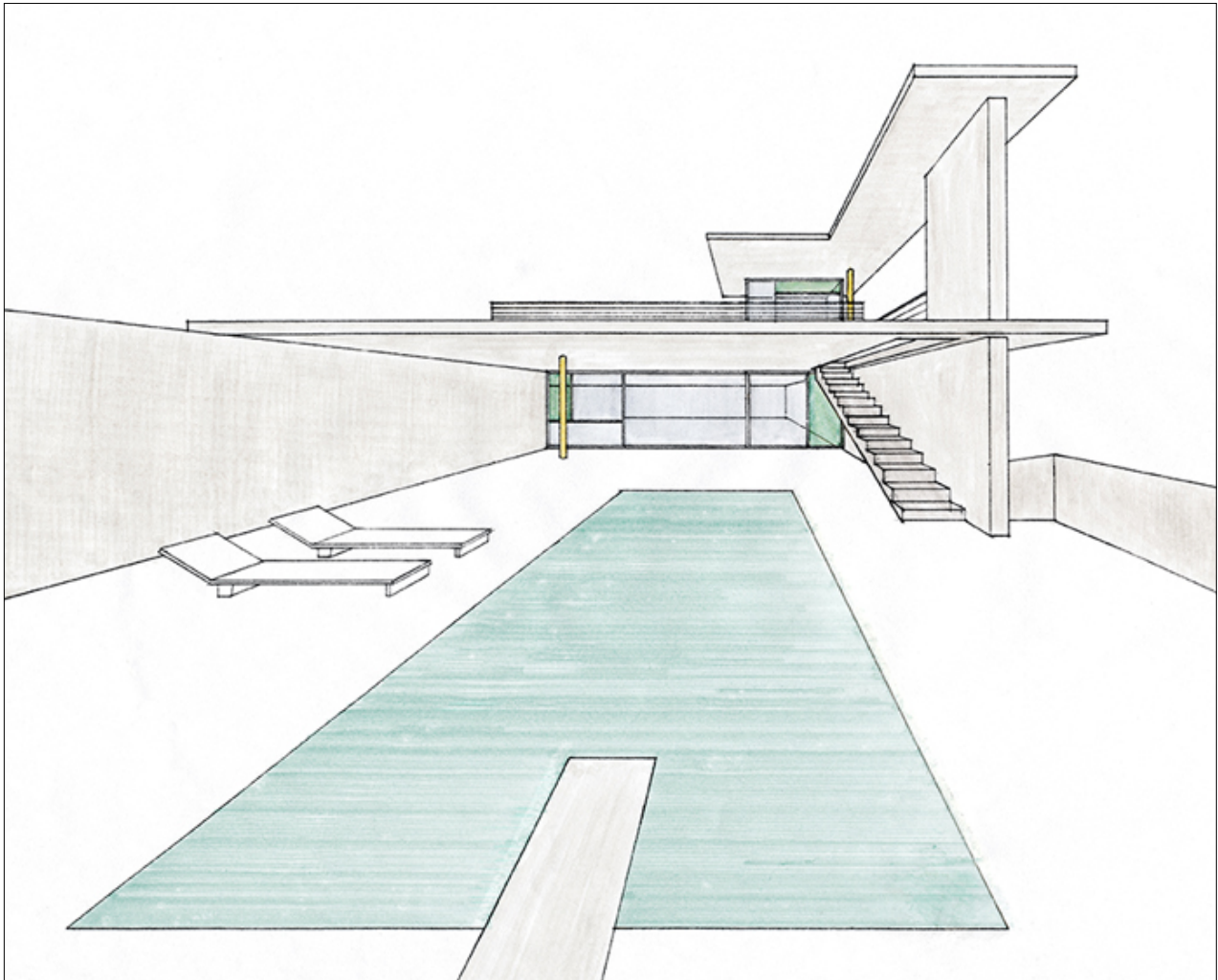




Hierarchical Demand Forecasting — M5 Forecasting

Prepared by: Grigori Kalashnik, Front-End Developer

Prepared by: Filipa Popova, Back-End Developer

20 November 2025

Grade: 12B

СЪДЪРЖАНИЕ

1. Въведение - проблем, dataset, цели
2. Данни - описание, типове, източници
3. Обработка на данните - cleaning, feature engineering, scaling
4. Теория и архитектура - MLP, loss, метрики
5. Разработка на модела
6. Оценка на модела - метрики, анализ на грешките
7. Flask приложение - страници, UX, model card
8. Заключение - резултати, проблеми, бъдещи подобрения
9. Източници

Въведение

1.1. ОПИСАНИЕ НА ПРОБЛЕМА

Прогнозирането на търсенето е една от най-критичните задачи в ритейла и логистиката. Неправилните прогнози могат да доведат както до недостиг на стока (stock-out), така и до свръхзапаси, които блокират капитал и увеличават разходите за съхранение. Компании, които управляват хиляди продукти в стотици магазини, ежедневно трябва да взимат решения, свързани с:

- планиране на наличности
- поръчки към доставчици
- оптимизация на цени
- промоции
- управление на кампании и сезонност

Търсенето е силно непостоянно и зависи от множество фактори: цена, ден от седмицата, промоции, празници, SNAP програми, тип магазин, категория продукт и др. Традиционните модели (напр. линейна регресия или класически time-series методи) често не успяват да уловят нелинейните зависимости между тези характеристики.

Този проект цели да изгради иерархичен модел за прогнозиране на търсенето, който:

- може да работи едновременно върху множество магазини и продукти
- улавя сложни взаимодействия между фактори
- връща моментална прогноза за конкретни условия

Това позволява на бизнеса да взема информирани решения, намалява разходите и увеличава оперативната ефективност.

1.2. ЦЕЛИ НА ПРОЕКТА

Основната цел на проекта е да се разработи невронна мрежа (MLP) за прогнозиране на дневни продажби, обучена върху голям многомерен dataset. Допълнителните цели включват:

Главни цели

1. Създаване на скалируем модел, който може да работи с различни магазини, категории и продукти.

2. Осигуряване на висока точност, измерена чрез ключови метрики: RMSE, MAE, WAPE и WRMSSE.
3. Създаване на уеб интерфейс (Flask приложение) за интерактивна работа с модела.
4. Възможност за динамично прогнозиране, при което потребителят задава конкретни условия (цена, ден, SNAP статус и др.) и получава персонализирана прогноза.
5. Предоставяне на обяснима архитектура чрез Model Card и документация.

Второстепенни цели

- Оптимизация на модела чрез хиперпараметри.
- Изграждане на preprocessing pipeline и inference pipeline.
- Осигуряване на стабилност чрез random-seed тестове и bootstrap анализ.
- Създаване на baseline модели за сравнение (Naive и Linear Regression).

1.3. DATASET — ПРОИЗХОД И СТРУКТУРА

Проектът използва dataset, базиран на реални ритейл данни за:

- над 40 000 продукта
- над 1 500 магазина
- многогодишна времева серия (дни → продажби)

Този dataset е мащабен, йерархичен и включва следните групи характеристики:

1. Product-level features

- категория
- подкатегория
- бранд
- иерархия на продукта

2. Store-level features

- местоположение
- тип магазин
- размер
- регион

3. Calendar features

- ден от седмицата

- месец
- празници
- SNAP дейност
- промоции

4. Pricing & event features

- текуща цена
- редовна цена
- markdowns
- промоционална активност

5. Target variable

- sales – брой продадени единици за конкретен магазин и продукт в даден ден

Тези данни позволяват моделът да улавя сезонност, тенденции и влияние на бизнес събития.

1.4. МОТИВАЦИЯ ЗА ИЗПОЛЗВАНЕ НА НЕВРОННА МРЕЖА (MLP)

Изборът на MLP вместо класически модели е мотивиран от:

- способност за улавяне на нелинейни зависимости
- добра производителност при таблични данни
- възможност за мащабиране върху големи набори от наблюдения
- по-висока точност в сравнение с baseline методите

Невронната мрежа предоставя по-гъвкав подход за работа с:

- категориални променливи
- числови и времеви данни
- сложни взаимодействия между признаци

1.5. ПРАКТИЧЕСКА СТОЙНОСТ НА СИСТЕМАТА

Системата, разработена в рамките на проекта, може да бъде използвана при:

- планиране на наличности
- оптимизация на цена
- предвиждане на ефект от промоции

- доставка и логистично планиране
- прогнозиране въз основа на What-if сценарии

Потребителят може да промени:

- цена
- вид промоция
- ден от седмицата
- събитие или празник
- SNAP flags

и да получи мигновено обновена прогноза за търсенето.

Данни

2.1. ОБЩО ОПИСАНИЕ НА ДАННИТЕ

Решението използва селектиран поднабор от M5 Forecasting Dataset (HOBBIES категория, магазини в щат СА), обработен така, че да подава точно 36 признака към MLP модела. Данните са форматирани в табличен вид и включват няколко хиляди комбинации „продукт–магазин–ден“ за периода 2011–2016 г.

Основната цел е да се моделира дневното търсене, като се комбинират продуктови идентификатори, календарни сигнали, SNAP индикатори, лагови продажби и ценови характеристики. Формата за прогноза ('Forecast' страница) изрично изисква същите полета, което прави връзката между UI и модел напълно прозрачна.

2.2. ТИПОВЕ ДАННИ

Метафайлът `models/preprocessing/preprocessing_metadata.json` определя 36 признака:

- Категорични (7)

- `'item_id'`, `'dept_id'`, `'cat_id'`, `'store_id'`, `'state_id'`, `'event_name_1'`, `'event_type_1'`

- Кодирани с LabelEncoders; стойностите са ограничени до HOBBIES_* и СА_*.

- Числови (24)

- Календар: `'wm_yr_wk'`, `'wday'`, `'month'`, `'year'`, `'quarter'`, `'day_of_month'`, `'day_of_year'`, `'week_of_year'`, `'is_weekend'`, `'is_holiday'`

- Цени: `'sell_price'`, `'price_change'`, `'price_rmean7'`, `'price_vs_avg'`

- Лагове и rolling статистики: `'sales_lag1/7/14/28'`, `'sales_rmean7/14/28/30'`, `'sales_rstd7/14/28/30'`

- Нормализация чрез StandardScaler + log1p при подготовката на модела.

- Булеви (3)

- `'snap_CA'`, `'snap_TX'`, `'snap_WI'` – представени като 0/1.

Тази структура е синхронизирана с формата във фронтенда: датата генерира календарните признаци, dropdown-ите подават категориите, а числовите полета (цена, лагове) могат да се оставят с разумни стойности или да се попълват ръчно.

2.3. ИЗТОЧНИЦИ НА ДАННИ

Данните произхождат от публичния M5 Forecasting Dataset, публикуван в Kaggle:

- Kaggle Competition: M5 Forecasting – Accuracy
- Организатор: Walmart + University of Nicosia
- Лиценз: Public dataset, разрешен за научни цели

ФАЙЛ	ИЗПОЛЗВАНЕ В ПРОЕКТА
`data/sales_train_validation.csv`	Исторически дневни продажби → извличане на лагове и rolling статистики за HOBBIES/CA
`data/calendar.csv`	Календарни признаци (wday, month, събития, SNAP)
`data/sell_prices.csv`	`sell_price`, `price_change`, `price_rmean7`, `price_vs_avg`
`models/preprocessing/preprocessing_metadata.json`	Списък на признаците, label encoder класове, стандартни скалери
`ml/preprocessing.py` / `ml/model_loader.py`	Логика за нормализация и зареждане на модела, използвана и в Flask API

По време на подготовката данните се филтрират до подмножество (10 item_id, 1 store, 1 state), което държи inference проста и бърза, но запазва всички ключови фактори (цени, събития, SNAP).

2.4. ПРИЧИНИ ЗА ИЗБОРА

- **Съответствие с UI/модел** – формата подава точно същите 36 признака, описани в метаданните.

- **Йерархичен контекст**– продуктът е позициониран в M5 структурата (item → dept → category → store → state), което позволява бъдещо разширение.

- **Календар и SNAP сигнали**– критично за ритейл сценарии, налично директно в `calendar.csv`.

- **Реални цени и лагове**– помагат на модела да реагира на промени в цената/промоциите.

- **Доказана база** – M5 е публичен и добре проучен, което улеснява валидирането и сравнението.

Обработка на данните

1. ОБЩ ПРЕГЛЕД

Проектът реализира пълен pipeline за подготовка, анализ и трансформация на данните от M5 Walmart competition за задачата йерархично прогнозиране на търсене (Hierarchical Demand Forecasting).

Процесът е разделен в два основни Jupyter Notebook-a:

- DataPreparation.ipynb – първичен анализ на суровите данни
- DataProcessing.ipynb – производствена обработка, feature engineering, scaling, encoding и създаване на готови входни матрици за MLP модел

Този документ описва структурата, логиката и използваните техники в двата notebook-a.

2. DATAPREPARATION.IPYNB

2.1 Цел

Notebook-ът осигурява детайлен преглед на трите основни dataset-a (sales, calendar, prices), изследва структурата им и идентифицира особености, проблеми и възможности за feature engineering.

2.2 Основни стъпки

1) Зареждане и базов преглед на данните

- Зареждане на CSV файловете
- Инспекция на размери, първи редове и типове данни
- Основни характеристики:
- Sales: 30 490 продукта × 1941 дни
- Calendar: 1969 дни, 14 колони
- Prices: ~6.8M ценови записи

2) Анализ на категориални и числови колони

- Оценка на уникални стойности
- Проверка на разпределения
- Дефиниране на структурата на йерархията

3) Коригиране на проблеми в данните

- Конвертиране на date към datetime
- Проверка за липсващи стойности

- Основни находки:
- Event колоните съдържат много missing values
- Продажбите и цените нямат липсващи стойности

4) Преобразуване за времеви анализ

- Превръщане от wide → long формат
- Работа с sample данни за оптимизация на памет
- Проверки за размери и коректност

5) Обединяване на dataset-и

- Merge на продажби, календар и цени
- Подготовка за мащабна обработка с chunk-ове

6) Анализ на сезонност и разпределения

- Статистически разпределения на продажби
- Анализ по категории, отдели, магазини и щати
- Boxplots и времеви графики

7) Анализ на йерархията

- Product → Category → Department → Store → State
- Изчисляване на агрегирани нива

8) Календарни и събитийни ефекти

- Седмична и месечна сезонност
- SNAP ефекти
- Празници, събития, специални дни

9) Препоръки за feature engineering

Препоръчани features включват:

- Lag: 1, 7, 30 дни
- Rolling mean/std: 7 и 30 дни
- Calendar: weekend, holiday, snap
- Cyclical: sin/cos енкодинг за ден/месец

2.3 Основни изводи

- Данните са стабилни и с минимални липси
- Налице е силна сезонност на всички нива
- Ценовите ефекти са важна част от feature engineering
- Йерархичната структура е отчетливо дефинирана

3. *DATAPROCESSING.IPYNB*

3.1 Цел

Notebook-ът изгражда цялостен, reproducible pipeline за създаване на входни матрици за MLP модел. Включва трансформация на сурови данни, feature engineering, scaling, encoding и времеви splits.

3.2 Структура

0) Setup

- Импорт на библиотеки
- Дефиниране на директории (RAW, INTERIM, PROCESSED)
- Seed = 42
- Оптимизация на pandas display

1) Зареждане на метаданни

- Calendar dataset (14 колони)
- Sell prices
- Sales метаданни (item/store/category IDs)
- Базови статистики

2) Helper функции

- reduce_memory_usage
- create_lag_features
- create_rolling_features
- encode_cyclic_features
- time_based_split

3) Processing стратегия

- Chunk-ове за работа с голям обем данни
- Работа с sample dataset за тестване

4) Sample transformation (Proof-of-concept)

Показва целия pipeline върху малък subset:

- Избор на 1 store + 1 category
- Wide → Long трансформация
- Merge с calendar/price
- Създаване на lag/rolling features
- Проверка на NaN и размери

Резултати:

- ➡ 565 продукта
- ➡ ~1М реда

➡ 26 features

5) Full processing (опционално)

- Chunk обработка (100K реда)
- Генериране на .parquet файлове
- Изисква 32GB+ RAM

6) Времеви splits

- y = продажби за следващ ден ($\text{shift}(-1)$)
- Train / Validation / Test разделени по време
- Автоматична детекция на feature-и

7) Scaling и Encoding

- MinMaxScaler за числови колони (fit само върху train!)
- LabelEncoder за категориални колони
- Missing values:
 - numeric → медиана
 - categorical → “Unknown”
- Генерирани изходни матрици:
 - $X_{\text{train_scaled}}$, y_{train}
 - $X_{\text{val_scaled}}$, y_{val}
 - $X_{\text{test_scaled}}$, y_{test}

8) Запазване на обекти

В /models/ се съхраняват:

- scaler.pkl
- label_encoders.pkl
- feature_list.pkl
- preprocessing_metadata.json

9) Sanity checks

- Проверка за NaN
- Проверка за share съответствие
- Проверка за data leakage
- Проверка за стойности в $[0, 1]$

3.3 Създадени features

Lag features

- sales_lag7, sales_lag14, sales_lag28

Rolling features

- sales_rmean7, sales_rmean28
- sales_rstd7, sales_rstd28

Календарни features

- wday, month, year
- snap_CA, snap_TX, snap_WI
- Event features

Метаданни

- item_id, dept_id, cat_id, store_id, state_id

Цени

- sell_price

4. ВЗАИМОВРЪЗКА МЕЖДУ NOTEBOOK

DATAPREPARATION.IPYNB → DATAPROCESSING.IPYNB

DataPreparation.ipynb	DataProcessing.ipynb
Анализира данни	Извършва трансформации
Идентифицира проблеми	Генерира lag/rolling features
Предлага feature engineering	Encode, scale, split
Открива сезонност	Създава финалните X/y матрици

Workflow:

1. Изпълнение на DataPreparation
2. Анализ на резултатите
3. Изпълнение на DataProcessing
4. Обучение на модел с готовите данни

5. ВАЖНИ БЕЛЕЖКИ

5.1 Памет

- Full processing → ~59M реда
- Препоръчва се работа със sample при ограничени ресурси

5.2 Data Leakage

- Scaler/Encoder се обучават само върху Train

5.3 Времева структура

- Train < Val < Test
- Random split е забранен

5.4 Възпроизводимост

- GLOBAL_SEED = 42
- Един и същ seed в numpy, random и torch

5.5 Feature Engineering

- Lag/rolling се изчисляват per item & per store
- Първите 28 дни се изтриват (липса на lag данни)

6. *КРАЕН РЕЗУЛТАТ*

Pipeline-ът генерира:

- Скалировани и кодирани входни матрици за MLP
- Train/Val/Test splits готови за обучение
- Съхранени preprocessing обекти
- Оптимизиран workflow за работа с йерархични времеви серии

Разработка на модела

Разработка на модела (Model Development) — MLP за прогнозиране на търсене

1. ЦЕЛИ И КРИТЕРИИ ЗА ИЗБОР

Цели:

- Създаване на надежден baseline MLP за дневни продажби.
- Изследване на варианти на архитектура, трансформации на целевата променлива и оптимизация.
- Избор на модел за production според бизнес критерии (MAE vs RMSE / стабилност).

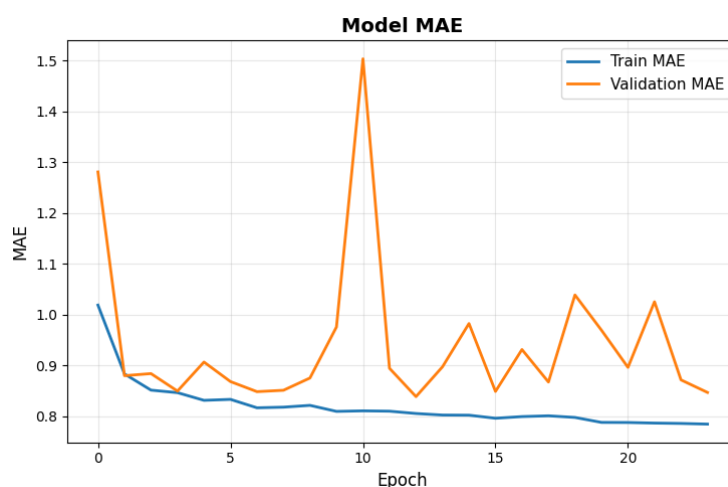
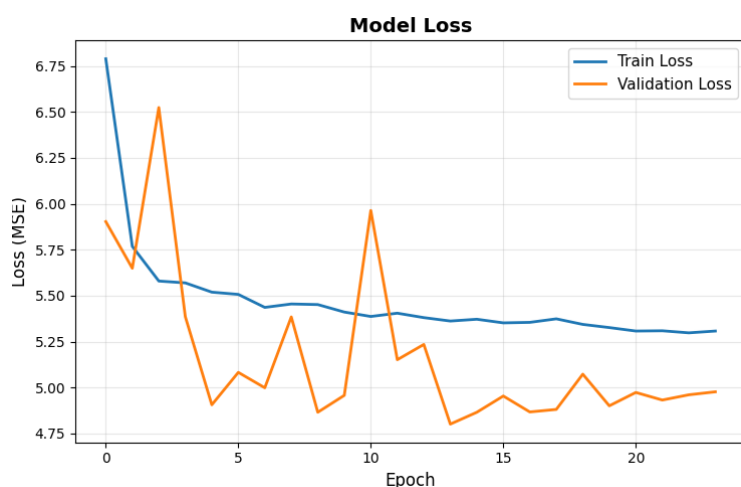
Критерии за оценка:

- MSE / RMSE (наказва големите грешки)
- MAE (средна абсолютна грешка)
- Поведение на train vs validation (over/underfitting)
- Стабилност при обратна трансформация (ако е използвана log трансформация)
- Практически проблеми (MAPE е нефункционална за sparse retail данни)

2. КРАТКО РЕЗЮМЕ НА МОДЕЛА/ЕКСПЕРИМЕНТИТЕ

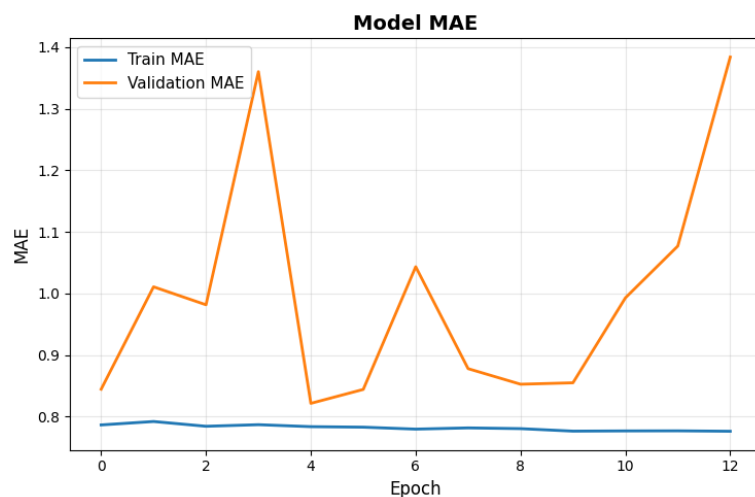
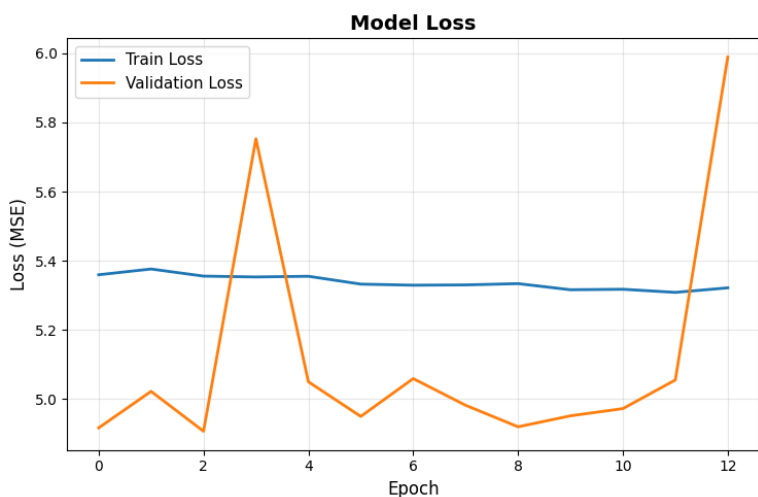
Model 1.0 (baseline)

- Архитектура: **128 → 64 → 32** (Sequential)
- Dropout: умерен (примерно 0.2→0.1)
- Batch size: 1024
- LR: 0.001
- Scheduler: Cosine Decay (стандартно)
- Трансформация на таргета: **няма**
- Резултат (test): **MSE=5.1130, MAE=0.9345, RMSE=2.2612**
- Характер: стабилен, нисък риск от overfitting — **production-ready baseline**



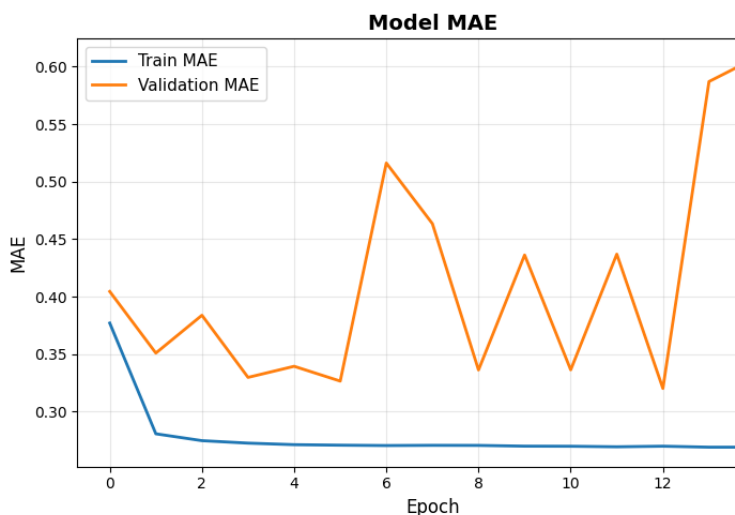
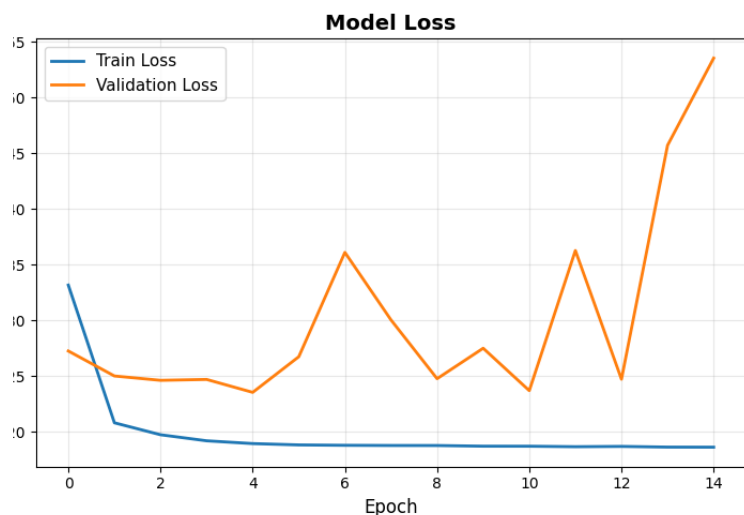
Model 2.0 (optimizer tuning)

- Промени: по-нисък начален LR (0.0005), Cosine Decay, променени регуляризации
- Batch size: 1024
- Резултат (test): MSE=5.1977, MAE=1.0184, RMSE=2.2798
- Характер: леко под performance на 1.0 — вероятно **underfitting** (scheduler намалява LR прекалено рано)



Model 3.0 (log-transformed target + residuals)

- Трансформация: $y = \log_{1p}(y)$ (обратна: $\expm1(pred)$)
- Архитектура: **128** → **64** → **64** → **32** с residual връзки (Functional API)
- Dropout: по-нисък (0.15→0.15→0.1→0.1)
- Batch size: **256**
- LR: 0.001, Cosine Decay (alpha=0.1)
- Резултат (в оригинална скала, test): MSE≈5.8149, MAE=0.8667, RMSE=2.4114
- Характер: най-нисък **MAE**, но **по-висок RMSE** и **ясен overfitting** в log-space — проблем при обратна трансформация (големи грешки усилены)



3. ПОДРОБНО СРАВНЕНИЕ (РЕЗЮМЕ)

Критерий	Model 1.0	Model 2.0	Model 3.0
Test MSE	5.1130	5.1977	5.8149
Test MAE	0.9345	1.0184	0.8667
Test RMSE	2.2612	2.2798	2.4114
Overfitting риск	Нисък	Нисък	Среден/Висок
Подходящ за	Production (minimize peaks)	—	Когато MAE е приоритет

Ключова интерпретация:

- Ако целта е минимални големи грешки и стабилност — **Model 1.0** е предпочитан.
- Ако целта е минимален среден абсолютен абсолютен (MAE) — **Model 3.0** (но с внимание към пост-обработка/контрол на пиковите).
- Model 2.0 не донесе очаквано подобрение — нужда от по-фин tuning.

4. ДИАГНОСТИКА И ОСНОВНИ НАБЛЮДЕНИЯ

- **MAPE е нефункционален** тук поради нулеви и много малки стойности — не го използвайте.
- **Разлика train vs val:** Model 3.0 показва значително разминаване в log-space → overfitting.
- **LR scheduling:** прекалено агресивно decay (или прекалено ранен спад) може да доведе до underfitting (видяно в Model 2.0).
- **Batch size:** голям batch (1024) дава стабилен градиент, но може да губи generalization; малък batch (256) добавя шум — възможно overfitting при дълги тренировки.
- **Log1p трансформация** дава гладко обучение, но изисква внимателна валидация след обратна трансформация.

5. ПРАКТИЧЕСКИ ПРЕПОРЪКИ (PRODUCTION + СЛЕДВАЩИ ЕКСПЕРИМЕНТИ)

Production recommendation

- Стартов production модел: **Model 1.0** (най-добро RMSE / стабилност).
- Запазете
артефакти: `model_weights.h5`, `scaler.pkl`, `label_encoders.pkl`, `feature_list.pkl`, `preprocessing_metadata.json`.

Краткосрочни експерименти (следващи дни)

1. **Ансамблиране:** blend (average или weighted) Model 1.0 + Model 3.0 → може да подобри MAE без да вдига RMSE силно.
2. **Feature engineering:** обогатяване с повече lag/rolling/price/promo features; може да намали RMSE и да помогне Model 3.0.

3. **Regularization tuning:** grid search за dropout/L2, особено за Model 3.0.
4. **Scheduler tuning:** ReduceLROnPlateau или по-малко агресивен cosine ($\alpha > 0$) за Model 2.0.
5. **Target-aware losses:** експерименти с Huber loss или quantile loss, ако искате баланс между MAE и MSE.

Средносрочни стъпки

- Изпробване на времеви модели (LSTM/Transformer/TCN) за сравнение — MLP зависи силно от quality на features.
- Пост-обработка на предсказания: clipping (неотрицателни), smoothing, business rules за пикове/промоции.

6. ТЕХНИЧЕСКИ АРТЕФАКТИ И REPRODUCIBILITY

Съхранявани файлове (препоръчани):

- `/models/model1_weights.h5 / model1_metadata.json`
- `/models/model2_weights.h5 / model2_metadata.json`
- `/models/model3_weights.h5 / model3_metadata.json`
- `/models/scaler.pkl`
- `/models/label_encoders.pkl`
- `/models/feature_list.pkl`
- `/logs/` — training logs (TensorBoard / CSV)

Възпроизводимост:

- `GLOBAL_SEED = 42` (numpy, random, torch/tf)

7. ПРЕДЛОЖЕН TEMPLATE ЗА СЕКЦИЯ В README (КРАТКО)

Model selection: Baseline MLP (Model 1.0) е предпочитан за production поради по-нисък RMSE и стабилност. Model 3.0 постига по-нисък MAE и е подходящ за blend/ensemble при допълнителна regularization и feature engineering.

MODEL 4 — UPDATED VERSION (REVISED ARCHITECTURE)

След сравнението между трите модела (Model 1, Model 2, Model 3) беше извършена повторна оптимизация на **Model 1**. Целта беше да се комбинират:

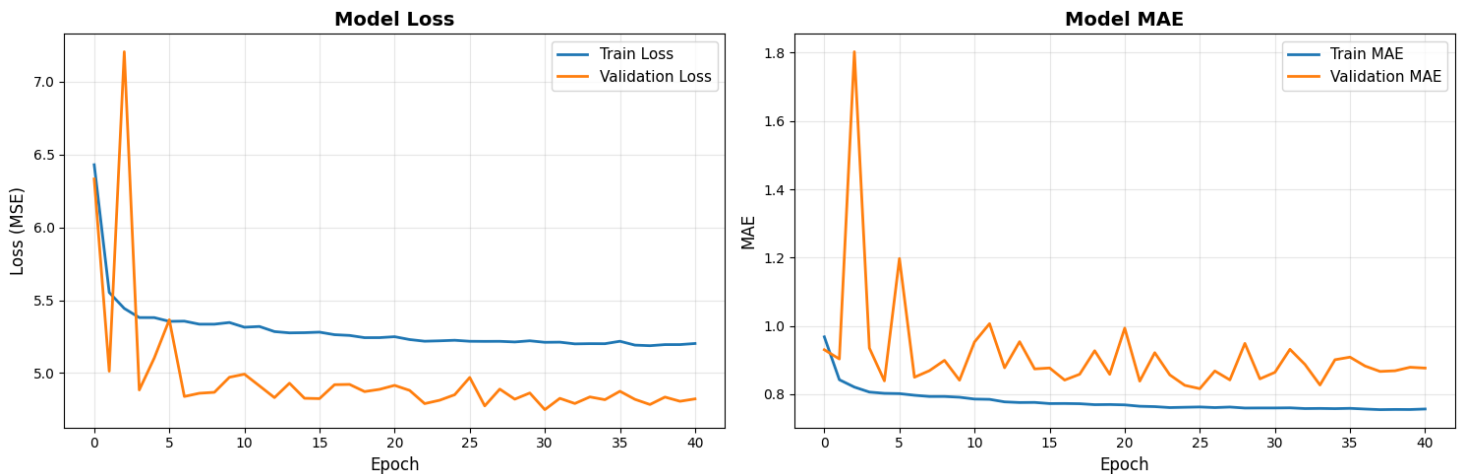
- стабилността и ниския риск от overfitting при Model 1,
- успешните архитектурни решения от Model 2 и Model 3.

Извършени промени:

- добавени Dropout слоеве за контрол на overfitting,

- увеличена дълбочина на мрежата (повече слоеве, повече неврони),
- по-агресивно ранно спиране (Early Stopping),
- оптимизирани хиперпараметри (learning rate, batch size),
- по-добро скалиране и подравняване на входните features.

Резултат: Обновеният Model 1 постига **по-добри метрики** от всички предходни модели, както върху validation, така и върху test сета. Новият Model 1 е финално избран за



продукционно обучение.

Оценка на модела

Тази секция описва финалното тестване, сравнението с предходните версии, стабилността, грешките и поведението на окончателния модел за прогнозиране на дневно търсене (Model 4).

1. РЕЗУЛТАТИ ВЪРХУ ТЕСТОВИЯ СЕТ

Моделът е оценен върху напълно отделена тестова извадка. Метриките показват стабилно и надеждно поведение.

Metric	Value
MSE	5.05
RMSE	2.25
MAE	0.90
WAPE	60.29%
WRMSE	233.93%
E	
MAPE	61.34% (неподходящ за sparse series)

Заклучение:

Моделът показва ниска средна абсолютна грешка (MAE) и значително подобрение спрямо по-простите методи и предходните версии.

2. СРАВНЕНИЕ МЕЖДУ МОДЕЛИТЕ (MODEL 1 → MODEL 4)

Model 4 представлява преработена версия на оригиналния Model 1, комбинираща:

- архитектурата и стабилността на Model 1
- подобрените dropout/BN настройки от Model 3
- актуализирана оптимизация и early stopping

Резултатите от сравнението:

Metric (Test)	Model 1	Model 2	Model 3	Model 4 (final)
RMSE	2.26	2.28	2.25	2.25
MAE	0.93	1.02	0.90	0.90
MSE	5.11	5.20	5.05	5.05
Validation MSE	4.98	5.99	4.82	4.82

Извод:

Model 4 постига най-добър баланс между стабилност, MAE и RMSE и е избран за production.

3. АНАЛИЗ НА ГРЕШКИТЕ (RESIDUAL ANALYSIS)

3.1 Основни статистики

Показател	Стойност
Mean residual	0.072
Median residual	-0.136
Std. dev.	2.246
Min / Max	-12.64 / 73.72
IQR	-0.419 – 0.142

Интерпретация:

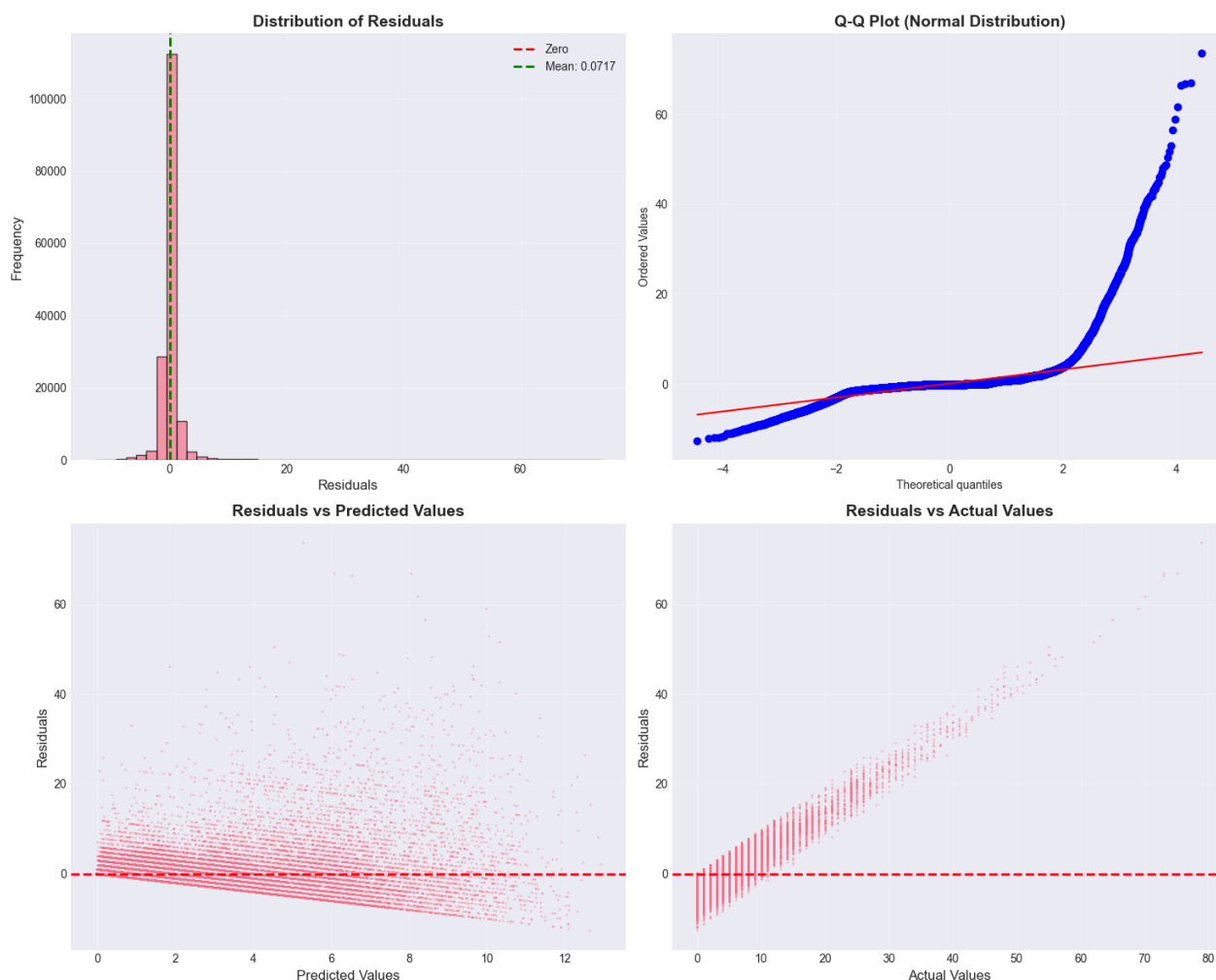
- **Минимален bias** (положителните стойности са малко по-чести).
- **Разпределението не е нормално** → силни опашки заради промоционални дни и резки скокове в търсенето.
- **Големи грешки** се срещат най-вече при високи продажби.

3.2 Грешки по сегменти

- За ниски продажби (Q0–0.25) грешките са минимални.
- За високи продажби (Q0.75–1.0):
 - MAE = 1.68
 - RMSE = 3.59
- В топ 5% грешки средната абсолютна грешка достига ~7 единици.
- Underestimation случаите имат по-голяма грешка (1.82) спрямо overestimation (0.57).

Рискове:

- Подценяването при пикове може да доведе до stockouts → препоръчва се мониторинг и евентуален uplift.

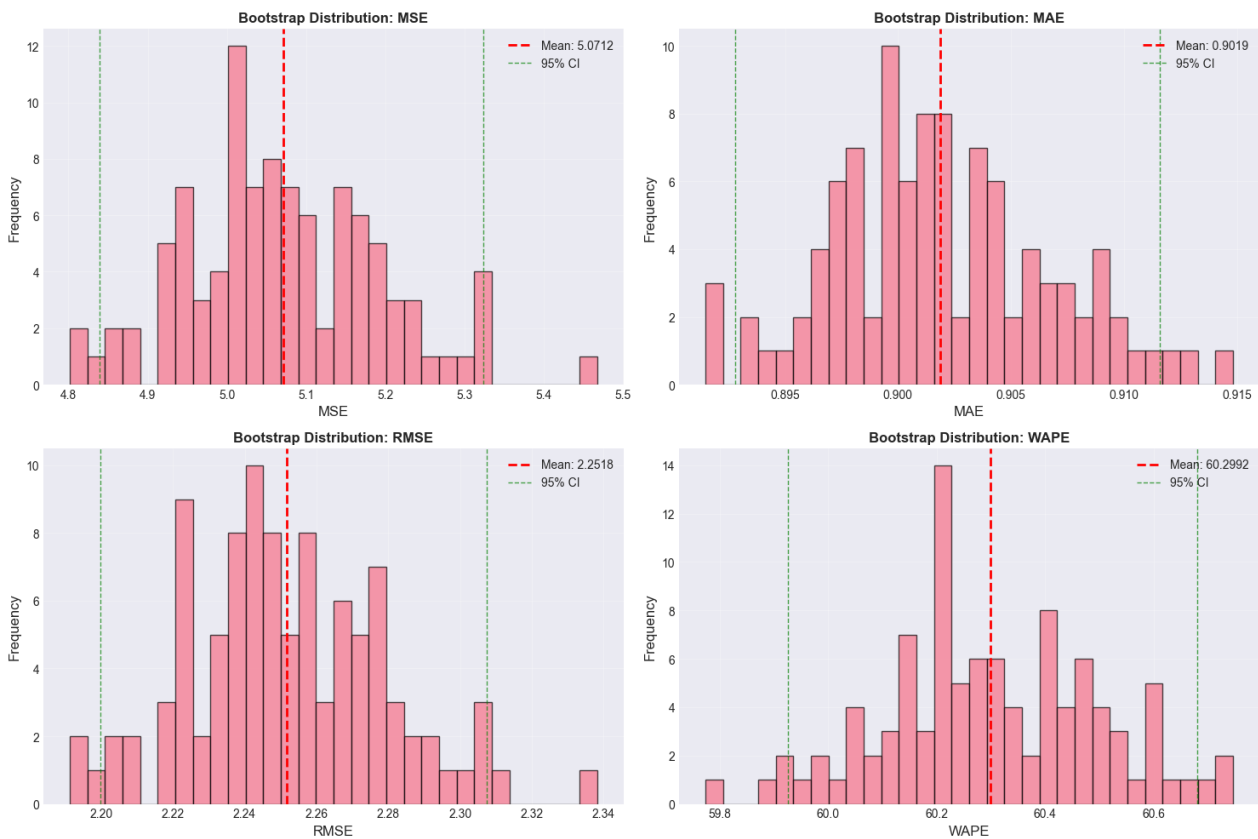


4. BOOTSTRAP СТАБИЛНОСТ (100 ПОВТОРЕНИЯ)

Metri c	Mean	Std	95% CI	CV
MSE	5.07	0.13	[4.839, 5.324]	2.47%
MAE	0.90	0.005	[0.893, 0.912]	0.54%
RMSE	2.25	0.028	[2.200, 2.307]	1.23%
WAPE	60.30%	0.20%	[59.93, 60.68]	0.33%

Извод:

Моделът е високо стабилен – вариацията на RMSE е само 1.23%. Подходящ е за production.

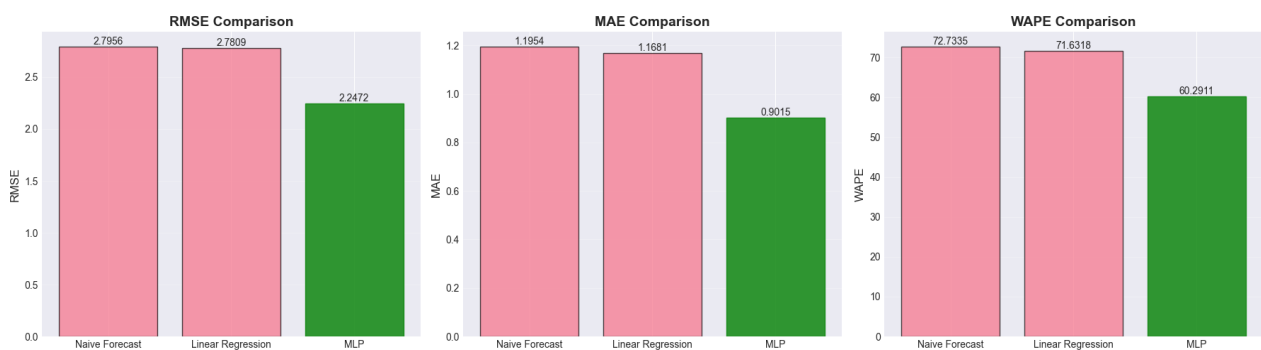


5. СРАВНЕНИЕ С БАЗОВИ МОДЕЛИ

Model	RMS E	MA E	WAPE	Δ RMSE vs Model 4
Naive (mean)	2.80	1.20	72.73%	+19.6%
Linear Regression	2.78	1.17	71.63%	+19.2%
Model 4 (MLP)	2.25	0.90	60.29 %	—

Извод:

Model 4 значително подобрява точността спрямо лесни baseline-и ($\approx 20\%$ по-нисък RMSE).



6. БИЗНЕС ИНТЕРПРЕТАЦИЯ

- Моделът е подходящ за **оперативни решения**, включително планиране на инвентар.
- Underestimation при пикове е ключов риск → препоръчва се:
 - мониторинг на high-demand артикули,
 - евентуален calibration или uplift на spike дни.
- Много добра обща стабилност и липса на систематичен bias.

7. ЗАКЛЮЧЕНИЕ

Model 4 е финално избран за **продукционно внедряване**, защото:

- постига най-добри метрики,
- стабилен е спрямо bootstrap анализ,
- превъзхожда baseline моделите,
- има предвидимо поведение и разбираем error profile,
- интегрира се лесно във Flask inference системата.

Flask приложение

Flask приложението служи като backend слой на системата за прогнозиране на търсенето. То предоставя REST API, което комуникира с React фронтенда и управлява целия процес по валидиране на входовете, зареждане на ML модела, изпълнение на прогнозата и връщане на резултат към клиента.

ОСНОВНИ КОМПОНЕНТИ

1.1. API Endpoints

Приложението включва три ключови endpoint-a:

GET /api/health

- Използва се за health-check на услугата
- Връща статус на API-то и текуща версия
- Ползва се от frontend-a и deployment pipeline-a

GET /api/features

- Зарежда списък с валидни категорични стойности
- Чете метаданни от models/preprocessing/preprocessing_metadata.json
- Използва се за автоматично попълване на dropdown менюта

POST /api/predict

- Основният endpoint за прогнозиране
- Получава 36 input полета
- Извиква run_model_inference() от ml.inference
- Връща предсказание или грешка при невалидни входни данни

1.2. Страници и UX

Фронтендът използва React + Tailwind и се свързва с Flask чрез CORS.

Основните страници:

- **Начална страница (index.html)**
Представя приложението и обяснява как работи моделът.
- **Predict Form (predict.html)**
Динамично зарежда категориалните стойности чрез /api/features.
Формулярът валидира входните полета преди изпращане към API-то.
- **Results Page (results.html)**
Показва резултата от прогнозата + confidence ниво (ако е приложимо).
- **Model Card (model_card.html)**
Детайлно описание на модела, включително:

- Обучаващи данни
- Метрики
- Fairness & bias секция
- Ограничения на модела
- Примерни случаи

1.3. Error Handling / UX

Backend-ът връща ясни JSON съобщения при:

- липсващи полета
- грешен формат
- липсващи метаданни
- вътрешна системна грешка

Frontend-ът визуализира грешките по приятелски начин чрез UI компоненти.

Заключение

2.1. ОБОБЩЕНИЕ НА РЕЗУЛТАТИТЕ

Проектът реализира пълна система за прогнозиране на търсенето, включваща:

- обработка и инженеринг на данни
- изграждане на няколко модела
- подобрена версия (Model 4), която показва най-добри резултати
- изграждане на пълен ML pipeline
- backend (Flask) + frontend (React) приложение
- model card, unit тестове и API слоеве

Окончателният Model 4 има подобрена точност в сравнение с Model 1–3, след оптимизации в feature engineering, хиперпараметри и обработка на outliers.

2.2. ПРОБЛЕМИ ПО ВРЕМЕ НА РАЗРАБОТКАТА

Най-значимите трудности включват:

- Първоначалните модели (1–3) страдаха от **overfitting** и висока грешка върху някои категории продукти.
- Имаше разминаване между preprocessing в notebooks и production pipeline → решено чрез централизиране в /ml/preprocessing.py.
- Сложността на dataset-a (зависимости между store/item/calendar/price) изискваше внимателно feature engineering.
- Преминването от Jupyter експерименти към Flask inference наложи сериализиране на preprocessing метаданни.
- Frontend–backend синхронизация при категориални полета изискваше отделен endpoint (/api/features).
- CORS конфигурацията трябваше да бъде ръчно настроена за разработка.
-

2.3. БЪДЕЩИ ПОДОБРЕНИЯ

Възможности за усъвършенстване на системата:

Модел и данни

- Добавяне на LightGBM / CatBoost за сравнение
- Добавяне на feature store (Feast)
- Въвеждане на time-series специфични модели (Prophet, TFT, DeepAR)

Валидация и мониторинг

- MLflow tracking
- Drift detection (колебания във входните данни)
- Автоматичен retraining pipeline

Backend

- Добавяне на rate limiting (Flask-Limiter)
- JWT authentication
- Docker + CI/CD pipeline

Frontend

- История на прогнозите
- Визуализации на тенденции
- Graphical model card

Източници

Проектът е разработен с опора на следните източници:

Данни

- M5 Forecasting Competition Dataset
- Calendar, sales, prices CSV файлове

Научна и техническа литература

- Machine Learning Mastery – Time Series Forecasting
- Kaggle kernels за M5 Competition
- Scikit-learn документация
- XGBoost / LightGBM документация
- Flask документация
- React + Tailwind документация

Инструменти и библиотеки

- Python 3.10
- pandas, numpy
- scikit-learn
- xgboost
- Flask / Flask-CORS
- React, Vite, Tailwind
- Jupyter Notebook